DAO

Test Unitarios en Python

Pruebas Automatizadas • Pytest • Unittest • Cobertura

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
DAO 2026

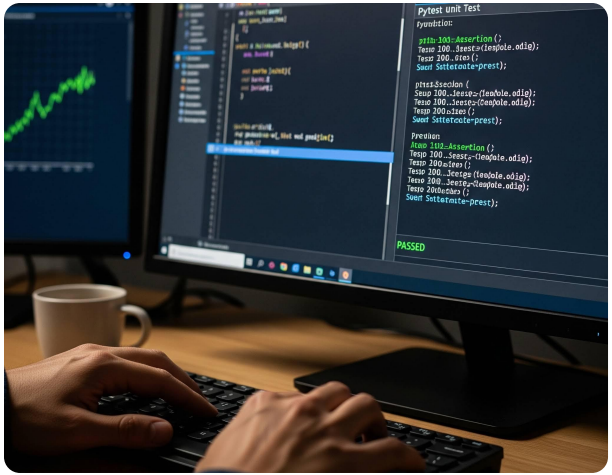
UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

{ assert }

```
def test_suma():
res = sumar(2, 3)
assert res == 5
```

Plantilla institucional

Testing Unitario con PyTest



Automatización de Pruebas

Asegurando la calidad, confiabilidad y mantenibilidad del código en Python.

01 ¿Qué es el Unit Testing?

02 Introducción a PyTest y sus ventajas

03 Estructura del proyecto para pruebas

04 Fixtures y Mocks en PyTest

05 Ejemplo práctico de pruebas unitarias

¿Qué es Unit Testing?

Conceptos Clave:

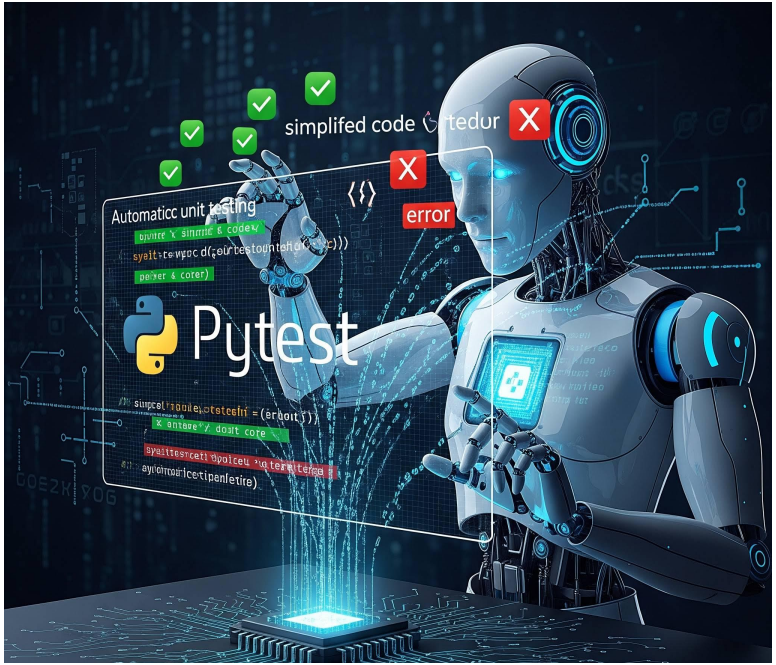
- **Verificación Individual:**
  Proceso de comprobación aislada de componentes de software.
- **Correcto Funcionamiento:**
  Asegura la respuesta esperada de cada unidad.
- **Detección Temprana:**
  Identifica errores en las primeras etapas de desarrollo.
- **Refactorización Segura:**
  Facilita la mejora del código sin alterar su comportamiento.
- **Calidad y Estabilidad:**
  Garantiza la solidez general de la aplicación.



Introducción a Pytest

¿Qué es Pytest y cuáles son sus ventajas?

- **Framework de Pruebas:** Framework principal para pruebas unitarias en Python.
- **Escritura Simplificada:** Simplifica la escritura y ejecución de tests.
- **Detección Automática:** Detección automática de pruebas en el proyecto.
- **Salidas Claras:** Genera reportes y salidas de error claras y concisas.
- **Instalación Vía Pip:** Instalación rápida mediante `pip install pytest`.



Estructura de un Proyecto

Organización y buenas prácticas

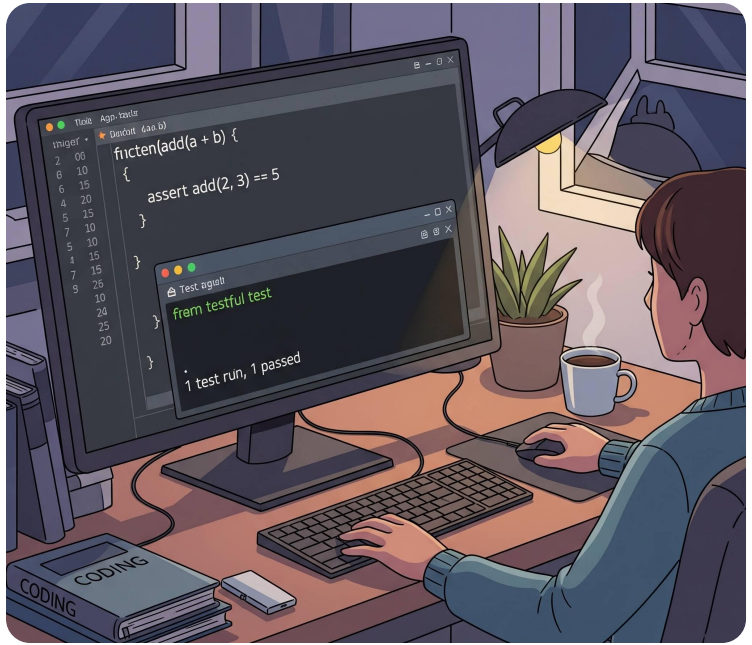
- **Organización de directorios:** Estructura clara separando código fuente y suite de pruebas.
- **Convención de nombres:** Estándar claro para la localización de archivos de prueba.
- **Funciones de prueba:** Nombres significativos que expresen el escenario evaluado.
- **Estructura con PyTest:** Distribución óptima para pruebas unitarias e integración.



Primera prueba con Pytest

Pasos para ejecutar la prueba:

- **Función Base:** Crear una función simple como ejemplo para probar.
- **Estructura:** Escribir el caso de prueba utilizando la sintaxis de Pytest.
- **Validación:** Usar 'assert' para verificar los resultados esperados.
- **Ejecución:** Ejecutar las pruebas directamente desde la línea de comandos.



Primera prueba con Pytest

1. Función a probar (módulo original)

```python
def sumar(a, b):
    return a + b
```

2. Test unitario con assert

```python
from src.mi_modulo import sumar

def test_sumar_dos_numeros():
    assert sumar(2, 3) == 5
```

Pytest ejecuta automáticamente las funciones que comiencen con **test_** y valida los resultados mediante la instrucción **assert**.



Tipos de Asserts en Pytest

Uso y verificación de afirmaciones en Pytest

Pytest utiliza la sentencia estándar **assert** de Python para validar que las condiciones evaluadas resulten verdaderas durante la ejecución.

- **Afirmaciones de verificación:** Permite comprobar el comportamiento esperado del código.
- **Variedad de 'asserts':** Evaluación de igualdad, pertenencia, identidad y comparaciones complejas.
- **Mensajes de error contextuales:** El mensaje de error de Pytest es muy útil y expresivo.
- **Reportes detallados:** Las pruebas fallidas proporcionan información detallada para facilitar la depuración.

Tipos de Asserts en Pytest

Igualdad

`assert a == b`
Verifica si dos valores son iguales.

`assert a != b`
Verifica si dos valores son diferentes.

Relacionales

`assert a > b`
Verifica si a es mayor que b.

`assert a < b`
Verifica si a es menor que b.

`assert a >= b`
Verifica si a es mayor o igual que b.

`assert a <= b`
Verifica si a es menor o igual que b.

Colecciones y Booleanos

`assert item in collection`
Verifica si un item se encuentra en una collection (lista, tupla, cadena, etc.).

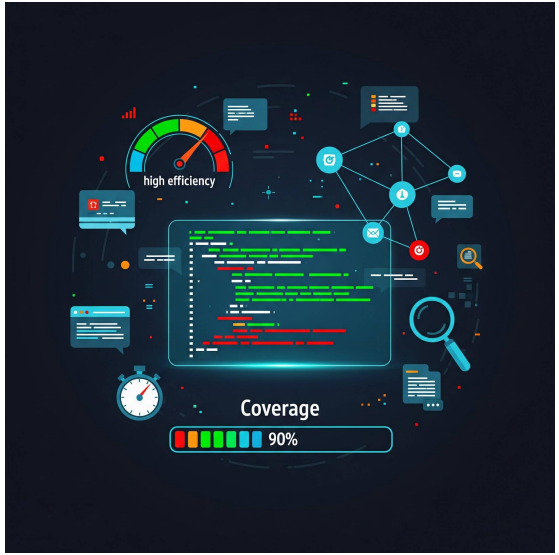
`assert item not in collection`
Verifica si un item NO se encuentra en una collection.

`assert boolean_expression`
Verifica si una expresión booleana es True.

Parámetros en Pytest

Ejecución con múltiples entradas

- **Parametrización:** Ejecuta una prueba con diferentes entradas.
- **@pytest.mark.parametrize:** Decorador para definir parámetros.
- **Casos de prueba:** Simplifica la escritura de múltiples casos de prueba.
- **Optimización:** Mejora la cobertura del código y la eficiencia.



Parámetros en Pytest

```python
import pytest
from figuras import Triangulo

@pytest.mark.parametrize(
    "lado1, lado2, lado3, expected_area, expected_perimeter",
    [
        (3, 4, 5, 6.0, 12),
        (5, 5, 8, 12.0, 18),
        (7, 10, 5, 16.25, 21),
    ]
)
def test_triangulo_parametrized(lado1, lado2, lado3, expected_area, expected_perimeter):
    t = Triangulo(lado1, lado2, lado3)
    assert round(t.superficie(), 2) == round(expected_area, 2)
    assert t.perimetro() == expected_perimeter

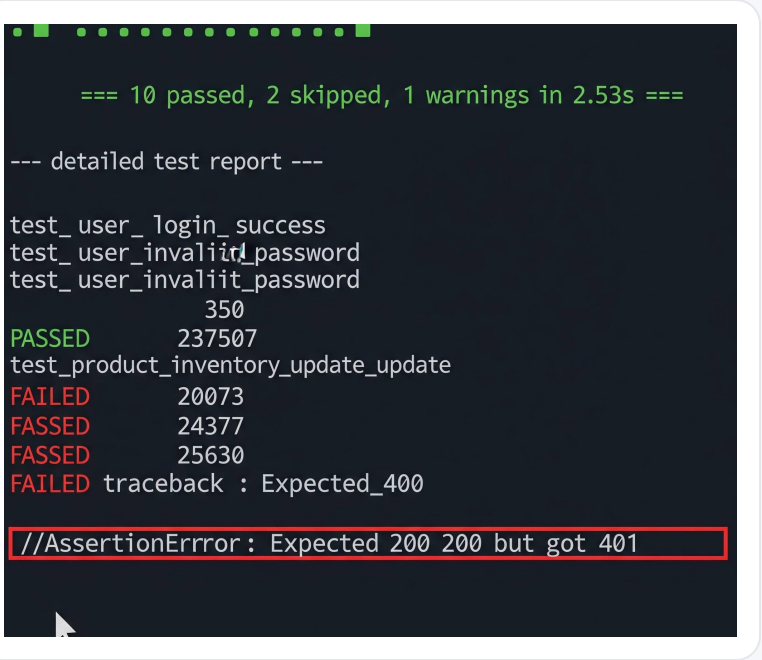
```

EJECUCIÓN Y REPORTES

Reportes PyTest

Uso de PyTest en consola

- **Línea de comandos:** Ejecuta pruebas con PyTest directamente desde la terminal.
- **Filtro por nombre:** Usa **pytest -k "nombre"** para ejecutar pruebas específicas.
- **Salida detallada:** Usa **pytest -v** para obtener un reporte más completo de los resultados.
- **Análisis de fallas:** Genera reportes estructurados para análisis detallado.



Conclusiones

Resumen del Aprendizaje

- **Simplicidad:** Pytest facilita el Unit Testing con simplicidad y flexibilidad.
- **Calidad del software:** Las pruebas unitarias mejoran la calidad del software.
- **Detección temprana:** Pytest agiliza la detección temprana de errores en el código.
- **Integración CI/CD:** Es valiosa la integración de Pytest con herramientas de CI/CD (Github, Gitlab, etc).
- **Automatización:** Automatiza la ejecución de pruebas para asegurar la calidad.



DAO

¡Gracias!

Hasta la próxima



UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA